



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of
Computing

SEMESTER 2/20252026

SPECIAL TOPIC IN DATA ENGINEERING (SECP3843)

SECTION 01

TUTORIAL IMAGE CLASSIFICATION USING CNN

LECTURER:

DR ARYATI BINTI BAKRI

NO	NAME	MATRIC NO
1	LIM YU HAN	A23CS0241
2	LING YU QIAN	A23CS0301
3	TEH RU QIAN	A23CS0191

1.0 Image Classification

1.1 Definition of Image Classification

Image classification is a fundamental task in the field of computer vision that involves automatically assigning a predefined label or category to an input image based on its visual content. The primary goal is to teach a computer system to recognize and categorize images in a manner similar to how humans perceive and identify objects in their environment. Mathematically, given an input image I with dimensions $H \times W \times C$ (height, width, and color channels), a classification model outputs a probability distribution over K possible classes, and the predicted class is the one with the highest probability.

1.2 The CIFAR-10 Dataset

This tutorial utilizes the CIFAR-10 dataset (Canadian Institute for Advanced Research), which contains 60,000 color images of size 32×32 pixels distributed across 10 balanced classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. The dataset is split into 50,000 training images and 10,000 testing images, providing a standardized benchmark for evaluating image classification models.

1.3 Applications of Image Classification

The importance of image classification extends across numerous real-world applications. In healthcare, classification models assist radiologists in detecting diseases from medical images such as X-rays, MRIs, and CT scans. In the automotive industry, autonomous vehicles rely on image classification to identify pedestrians, traffic signs, and other vehicles for safe navigation. Furthermore, security systems utilize facial recognition technology based on image classification for access control and surveillance while agricultural applications use leaf image analysis to detect crop diseases early. These diverse applications demonstrate why image classification remains an active and crucial area of research in artificial intelligence.

1.4 Challenges in Image Classification

Despite significant advances in deep learning, image classification faces several persistent challenges. Firstly, intra-class variation occurs when the same object appears differently due to changes in lighting conditions, viewing angles, scale, or pose. For example, a cat photographed from different angles or under different lighting can look dramatically different, making classification difficult. Second, inter-class similarity presents another challenge, where different objects share visual characteristics. Cats and dogs both have four

legs, fur, tails, and similar body shapes, while birds and airplanes can appear similar from certain angles due to their central body and two-wing silhouette. Third, background clutter, where objects are partially occluded or surrounded by distracting elements, further complicates feature extraction.

2.0 Convolutional Neural Network (CNN)

2.1 Definition of CNN

A Convolutional Neural Network (CNN) is a specialized class of deep neural networks designed specifically for processing grid-like data, with images being the most common example. Unlike traditional Artificial Neural Networks (ANNs) that flatten input images into one-dimensional vectors and lose all spatial relationships between pixels, CNNs preserve spatial hierarchies through two fundamental concepts including local connectivity and parameter sharing.

Local connectivity means that each neuron in a convolutional layer only connects to a small region of the input, allowing the network to focus on local patterns such as edges and textures. Parameter sharing means that the same filter is applied across the entire image, dramatically reducing the number of parameters while enabling the detection of features regardless of their position in the image. These properties make CNNs significantly more parameter-efficient than ANNs, as demonstrated by the fact that the baseline CNN in this tutorial uses approximately 282,000 parameters compared to the ANN's 9.2 million parameters while achieving substantially higher accuracy.

2.2 CNN Architecture Used in the Tutorial

The architecture of CNN consists of several types of layers working in sequence. Convolutional layers apply learnable filters to the input to produce feature maps, where each filter detects a specific type of feature such as horizontal edges, vertical edges, or color blobs. The Rectified Linear Unit (ReLU) activation function, defined as $f(x) = \max(0, x)$, is applied after each convolutional layer to introduce non-linearity into the network while mitigating the vanishing gradient problem and enabling faster convergence. Pooling layers, typically using max pooling operation, reduce the spatial dimensions of feature maps while retaining the most important information, which decreases computational complexity and provides translation invariance.

After several alternating convolutional and pooling layers, a flatten layer converts the multi-dimensional feature maps into a one-dimensional vector, which is then processed by

one or more fully connected (dense) layers that perform high-level reasoning and combine extracted features. Finally, the output layer uses the softmax activation function to produce a probability distribution over the 10 classes, and the class with the highest probability is selected as the model's prediction.

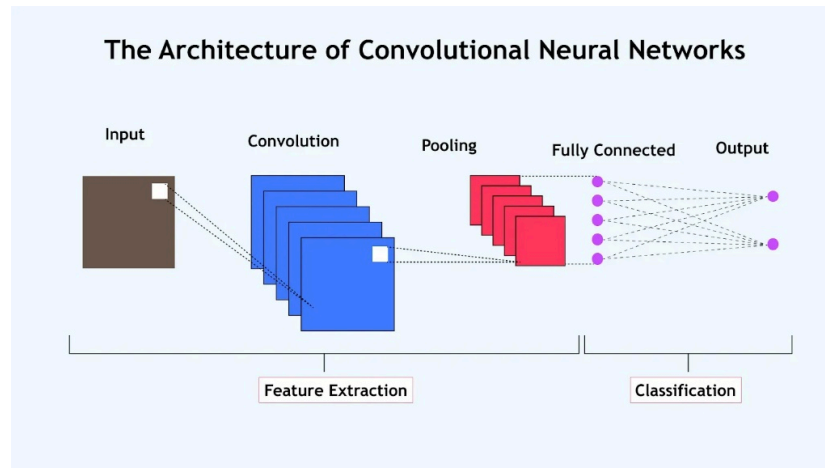


Figure 1: Standard structural pipeline of a Convolutional Neural Network (CNN), illustrating the transition from local feature extraction (convolution and pooling) to global class probability estimation via fully connected layers.

2.3 How CNN Learns Hierarchical Features

One of the most powerful aspects of CNNs is their ability to learn hierarchical feature representations automatically without manual feature engineering. Early convolutional layers detect low-level features such as edges, corners, and color blobs. Middle layers combine these simple features to recognize mid-level patterns such as textures, stripes, curves, and simple shapes. In addition, deeper layers assemble these patterns into high-level features representing object parts like eyes, wheels, wings, and tails. The final layers then integrate these object parts to recognize complete objects such as cats, cars, or airplanes.

This hierarchical learning occurs automatically during training as the network learns to combine simple features into progressively more complex representations, eliminating the need for hand-crafted features that dominated computer vision before the deep learning era.

3.0 Evaluation of CNN

Model evaluation is a critical step in the machine learning pipeline that assesses how well a trained model performs on unseen data, ensuring that the model has learned generalizable patterns rather than simply memorizing the training data. Without proper evaluation, a model

that performs excellently on training data may fail completely when deployed in real-world applications.

3.1 Data Splitting Strategy

To ensure fair evaluation, the CIFAR-10 dataset is split into distinct subsets, the training set (50,000 images) for learning model parameters and the test set (10,000 images) reserved exclusively for final evaluation after training is complete. The strict separation between training and test data is crucial because evaluation metrics calculated on the test set reflect the model's true generalization ability rather than its capacity to memorize training examples.

3.3 Evaluation Metrics

Multiple evaluation metrics are used to comprehensively assess model performance, each providing different insights into model behavior. Accuracy, defined as the proportion of correctly classified instances out of all instances, is the most intuitive metric but can be misleading for imbalanced datasets. Precision measures how many of the predicted positive instances were actually correct, making it important for applications where false positives are costly, such as spam detection where marking legitimate emails as spam is unacceptable. Recall measures how many of the actual positive instances were correctly identified, making it critical for applications where false negatives are costly, such as disease detection where missing a positive case could have serious consequences. The F1-score, which is the harmonic mean of precision and recall, provides a balanced single metric when both false positives and false negatives are important. The loss function, specifically categorical crossentropy for multi-class classification, quantifies the difference between predicted probability distributions and actual labels, with lower loss indicating better model calibration and confidence in predictions.

3.4 Experimental Results from This Tutorial

From the experimental results in this tutorial, the baseline CNN achieved a test accuracy of 68.38%, which is approximately 19 percentage points higher than the ANN's 49% accuracy. This substantial improvement demonstrates the fundamental advantage of convolutional architectures for image data, as CNNs preserve spatial relationships while ANNs destroy them through flattening. However, a 9.52% gap between training accuracy (77.9%) and test accuracy (68.38%) indicates the presence of overfitting, meaning the model has learned some training-specific patterns that do not generalize well to unseen data. This overfitting occurs

because the model has sufficient capacity (282,250 parameters) to memorize the training set, no regularization techniques such as dropout or batch normalization were applied, and data augmentation was not used to increase training data diversity. The confusion matrix analysis reveals that animal classes (bird, cat, dog) consistently underperform compared to vehicle classes (automobile, ship, truck). This class-level performance disparity occurs because cats and dogs share similar visual characteristics (four legs, fur, tails, similar body shapes) and the low 32×32 pixel resolution makes distinguishing features such as whiskers and ear shapes difficult to capture, while vehicles have more distinct geometric shapes and often contrast better with their backgrounds.

4.0 Steps hand on in Python

Step 1: Import Required Libraries

tensorflow.keras: Build and train deep learning models

numpy: Perform numerical operations

matplotlib: Display images and visualizations

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np
import matplotlib.pyplot as plt
```

Figure 4.1 Import Required Libraries

Step 2: Load CIFAR-10 Dataset

The dataset is downloaded and split into training images (50,000) and testing images (10,000).

```
(X_train, y_train), (X_test, y_test) = datasets.cifar10.load_data()
```

Figure 4.2 Load CIFAR-10 Dataset

Step 3: Examine Dataset Shape

Figure 4.3 shows the X_test dataset contains 10,000 color images. Each image has a resolution of 32 x 32 pixels and consists of 3 RGB channels.

```
X_test.shape
```

```
(10000, 32, 32, 3)
```

Figure 4.3 Examine X_text Dataset Shape

Figure 4.4 shows the X_train dataset contains 50,000 color images. Each image has a

resolution of 32 x 32 pixels and consists of 3 RGB channels.

```
X_train.shape
```

```
(50000, 32, 32, 3)
```

Figure 4.4 Examine X_train Dataset Shape

Figure 4.5 shows the y_train dataset stored as a column vector with 50,000 rows and 1 column.

```
y_train.shape
```

```
(50000, 1)
```

```
y_train[:5]
```

```
array([[6],  
       [9],  
       [9],  
       [4],  
       [1]], dtype=uint8)
```

Figure 4.5 Examine y_train Dataset Shape

Step 4: Reshape Labels

Figure 4.6 shows the y_train and y_test dataset were reshaped into 1D vectors to simplify processing and ensure compatibility with the CNN training and evaluation functions.

```
y_train = y_train.reshape(-1,)  
y_train[:5]
```

```
array([6, 9, 9, 4, 1], dtype=uint8)
```

```
y_test = y_test.reshape(-1,)
```

Figure 4.6 Reshape Labels

Step 5: Define Class Name

Figure 4.7 shows numerical labels mapping to meaningful category names.

```
classes = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
```

Figure 4.7 Define Class Name

Step 6: Create Function to Display Images

Figure 4.8 shows a function for displaying an image and its corresponding class label.

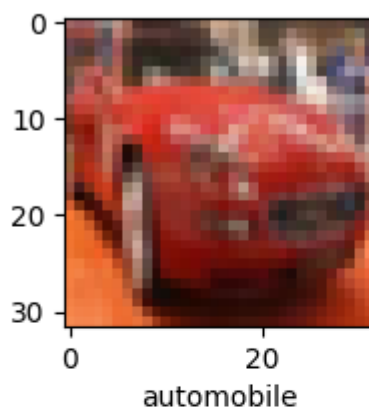
```
def plot_sample(X, y, index):  
    plt.figure(figsize=(15,2))  
    plt.imshow(X[index])  
    plt.xlabel(classes[y[index]])
```

Figure 4.8 Function that Display Images

Step 7: Visualize Sample Images

Figure 4.9 shows visual inspection 'plot_sample' helps verify that labels match the images.

```
plot_sample(X_train, y_train, 5)
```



```
plot_sample(X_train, y_train, 501)
```

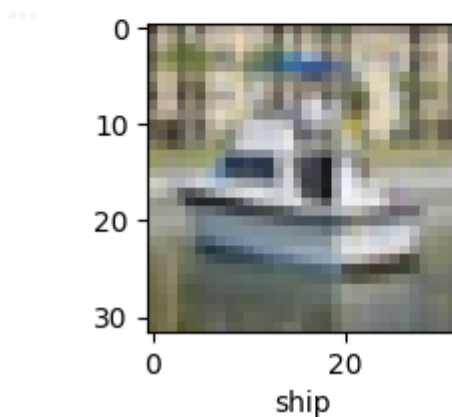


Figure 4.9 Visual Inspection

Step 8: Normalize Pixel Values

Figure 4.10 shows pixel values were normalized to a range of 0 to 1 by dividing 255. Normalization improves training efficiency by scaling the input data, resulting in faster training, better convergence and enhanced model performance.

```
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Figure 4.10 Normalize Pixel Values

Step 9: Build Artificial Neural Network (ANN)

The ANN Model consists of four layers shown as Figure 4.11:

Flatten Layer

CIFAR-10 images have a size of 32 x 32 pixels with 3 color channels (RGB). The Flatten layer converts the image from the 3D array into a 1D vector of length 3072 (32 x 32 x 3). This transformation is necessary because fully connected Dense layers accept 1D input vectors.

First Hidden Layer

This layer consists of 3000 neurons. The Rectified Linear Unit (ReLU) activation function is used: $f(x) = \max(0, x)$. ReLU helps the network learn complex patterns while reducing the vanishing gradient problems.

Second Hidden layer

This layer consists of 1000 neurons and further extracts important features from the image.

Output Layer

The output layer has 10 neurons which represent the 10 classes in the CIFAR-10 dataset. Then, the Softmax function converts outputs into probabilities whose sum equals 1. The class with the highest probability is selected as the prediction.

```
ann = models.Sequential([
    layers.Flatten(input_shape = (32,32,3)), #Flatten Layer
    layers.Dense(3000, activation = 'relu'), #First Hidden Layer
    layers.Dense(1000, activation = 'relu'), #Second Hidden Layer
    layers.Dense(10, activation = 'softmax') #Output Layer
])
```

Figure 4.11 Build Artificial Neural Network (ANN)

Step 10: Compile the ANN Model

Figure 4.12 shows Stochastic Gradient Descent (SGD) is used as its optimizer. SGD updates model weights during training to minimize prediction errors. Besides, the model uses sparse categorical crossentropy to measure how different the predicted outputs are from the actual labels. Loss function is suitable because the dataset is a multi-class classification problem and stored in integers. Accuracy is measured by the percentage of correctly classified images.

```
ann.compile(optimizer = 'SGD',  
            loss = 'sparse_categorical_crossentropy',    #Loss Function  
            metrics=['accuracy'])
```

Figure 4.12 Compile the ANN Model

Step 11: Train the ANN Model

The ANN gradually learned image patterns over 5 epochs. The final accuracy shown in figure 4.13 is 0.4986. This indicates that a simple ANN does not understand how pixels are arranged in an image, so it has difficulty recognizing image patterns and objects.

```
ann.fit(X_train, y_train, epochs=5)
```

```
... /usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: User  
    super().__init__(**kwargs)  
Epoch 1/5  
1563/1563 ————— 130s 80ms/step - accuracy: 0.3562 - loss: 1.8079  
Epoch 2/5  
1563/1563 ————— 124s 80ms/step - accuracy: 0.4261 - loss: 1.6223  
Epoch 3/5  
1563/1563 ————— 124s 80ms/step - accuracy: 0.4569 - loss: 1.5405  
Epoch 4/5  
1563/1563 ————— 123s 79ms/step - accuracy: 0.4817 - loss: 1.4797  
Epoch 5/5  
1563/1563 ————— 150s 84ms/step - accuracy: 0.4986 - loss: 1.4322  
<keras.src.callbacks.history.History at 0x7fc802f617f0>
```

Figure 4.13 Train the ANN Model

Step 12: Evaluate the ANN Performance

The model achieved an overall accuracy of 49%. Figure 4.14 shows the best-performing classes were Class 1 (automobile), Class 8 (ship) and Class 9 (truck), achieving F1-scores of 0.61, 0.62 and 0.56 respectively. In contrast, the poorest-performing classes were Class 2 (bird), Class 5 (dog) and Class 3 (cat), with F1-scores of 0.23, 0.34 and 0.39 respectively. Hence, ANN can recognize objects with clear shapes such as automobiles and ships, but struggles with animals because it cannot effectively capture image features.

```

from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred = ann.predict(X_test)
y_pred_classes = [np.argmax(element) for element in y_pred]

print("Classification Report: \n", classification_report(y_test, y_pred_classes))

```

```

313/313 ----- 9s 29ms/step
Classification Report:

```

	precision	recall	f1-score	support
0	0.51	0.60	0.55	1000
1	0.56	0.66	0.61	1000
2	0.50	0.15	0.23	1000
3	0.32	0.50	0.39	1000
4	0.43	0.42	0.43	1000
5	0.53	0.25	0.34	1000
6	0.51	0.55	0.53	1000
7	0.51	0.59	0.55	1000
8	0.64	0.60	0.62	1000
9	0.52	0.61	0.56	1000
accuracy			0.49	10000
macro avg	0.50	0.49	0.48	10000
weighted avg	0.50	0.49	0.48	10000

Figure 4.14 Evaluate the ANN Performance

Step 13: Confusion Matrix Visualization

Figure 4.15 shows the confusion matrix that determines which classes are frequently confused with one another and identifies areas where the model's performance can be improved.

```
import seaborn as sns
plt.figure(figsize=(14,7))
sns.heatmap(y_pred, annot=True)
plt.ylabel('Truth')
plt.xlabel('Prediction')
plt.title('Confusion Matrix')
plt.show
```

```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None

**Auto-show in jupyter notebooks**

The jupyter backends (activated via ``%matplotlib inline``,
``%matplotlib notebook``, or ``%matplotlib widget``), call ``show()`` at
the end of every cell by default. Thus, you usually don't have to call it
explicitly there.
```

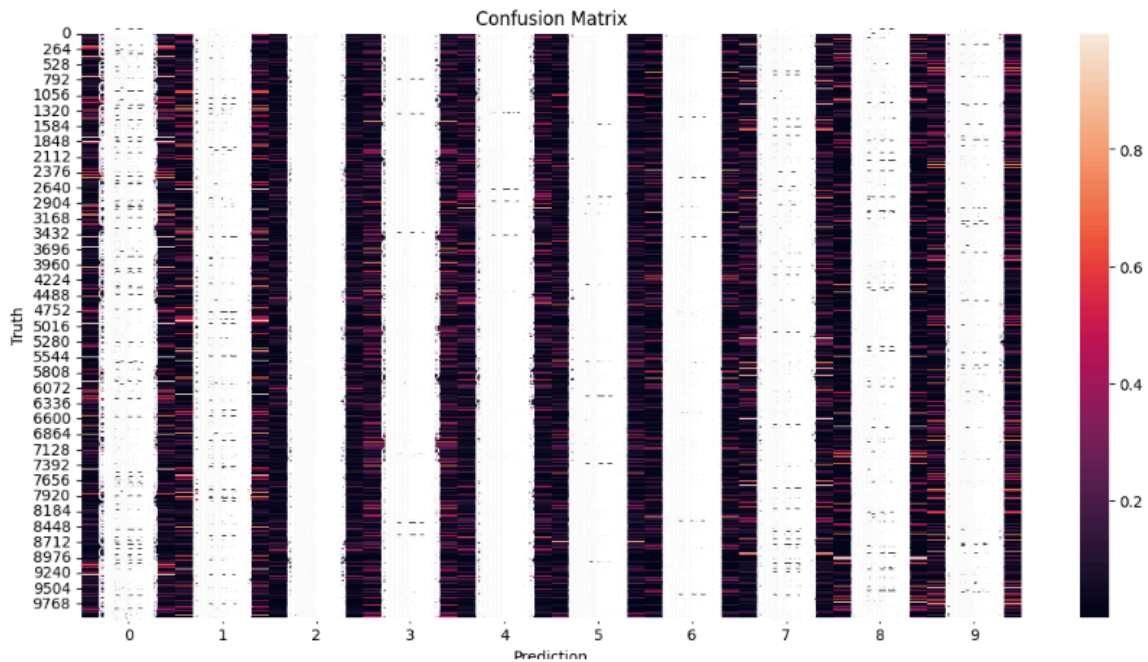


Figure 4.15 Confusion Matrix Visualization

Step 14: Build CNN Model

The CNN Model consists multiple layers shown as Figure 4.16:

Convolutional Layer 1

32 filters are applied to the input image to help the model detect basic features such as edges, textures and corners.

Max Pooling Layer

This layer reduces the size of the feature maps while retaining the most important information. This decreases computational complexity and helps reduce overfitting.

Convolutional Layer 2

64 filters are applied to learn more advanced features such as shapes, object parts and patterns.

Dense Layer

Learned features are combined to make predictions.

Output Layer

This layer produces probabilities for the 10 classes. The class with the highest probability is selected as the model's prediction.

```
cnn = models.Sequential([
    layers.Conv2D(filters=32, kernel_size=(3, 3), activation='relu', input_shape=(32, 32, 3)), #Convolution Layer 1
    layers.MaxPooling2D((2, 2)), #Max Pooling Layer

    layers.Conv2D(filters=64, kernel_size=(3, 3), activation='relu'), #Convolution Layer 2
    layers.MaxPooling2D((2, 2)),

    layers.Flatten(), #Flatten Layer
    layers.Dense(64, activation='relu'), #Dense Layer
    layers.Dense(10, activation='softmax') #Output Layer
])
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape` argument to `Conv2D` or `Conv3D` layers. It is deprecated and will be removed in Keras 3.0.0. Use `input\_shape` argument to `Model` instead.

Figure 4.16 Build CNN Model

Step 14: Train CNN Model

Figure 4.17 shows the CNN model had a steady improvement in accuracy throughout the training process. The training accuracy increased from 47.8% in Epoch 1 to 70.5% in Epoch 5 and reached 77.9% in Epoch 10. The final training accuracy was 77.9%, indicating that the CNN learned the image features effectively and performed significantly better over time.

```
cnn.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])
```

```
cnn.fit(X_train, y_train, epochs=10)
```

```
Epoch 1/10
1563/1563 ————— 69s 43ms/step - accuracy: 0.4781 - loss: 1.4531
Epoch 2/10
1563/1563 ————— 81s 43ms/step - accuracy: 0.6122 - loss: 1.1126
Epoch 3/10
1563/1563 ————— 67s 43ms/step - accuracy: 0.6538 - loss: 0.9902
Epoch 4/10
1563/1563 ————— 67s 43ms/step - accuracy: 0.6835 - loss: 0.9107
Epoch 5/10
1563/1563 ————— 82s 43ms/step - accuracy: 0.7054 - loss: 0.8500
Epoch 6/10
1563/1563 ————— 68s 43ms/step - accuracy: 0.7200 - loss: 0.7998
Epoch 7/10
1563/1563 ————— 67s 43ms/step - accuracy: 0.7365 - loss: 0.7536
Epoch 8/10
1563/1563 ————— 67s 43ms/step - accuracy: 0.7491 - loss: 0.7147
Epoch 9/10
1563/1563 ————— 71s 45ms/step - accuracy: 0.7636 - loss: 0.6760
Epoch 10/10
1563/1563 ————— 67s 43ms/step - accuracy: 0.7785 - loss: 0.6353
<keras.src.callbacks.history.History at 0x7fc74ffc8bc0>
```

Figure 4.17 Train CNN Model

Step 15: Evaluate CNN Model

Figure 4.18 shows the accuracy for CNN model is 0.6838 which is higher than ANN model.

```
cnn.evaluate(X_test, y_test)
```

```
313/313 ————— 4s 13ms/step - accuracy: 0.6838 - loss: 0.9665  
[0.9665185809135437, 0.6837999820709229]
```

Figure 4.18 Evaluate CNN Model

Step 16: Predict Class Probabilities

Figure 4.19 shows the predicted class probabilities produced by the CNN model. The highest probability in each row determines the final predicted class.

```
y_pred = cnn.predict(X_test)  
y_pred[:5]
```

```
313/313 ————— 4s 13ms/step  
array([[1.4584543e-03, 4.0731911e-04, 5.1789456e-03, 4.4850805e-01,  
        2.5405636e-04, 1.1264058e-02, 1.3371165e-02, 3.6525376e-05,  
        5.1847130e-01, 1.0501401e-03],  
       [6.9757726e-04, 5.9416336e-03, 7.9795234e-07, 2.9573556e-07,  
        2.5826068e-09, 5.2710375e-10, 3.7521559e-09, 3.0031664e-08,  
        9.8805267e-01, 5.3069415e-03],  
       [8.5548395e-03, 2.1642705e-02, 2.9409351e-04, 3.8977561e-04,  
        5.4024975e-05, 1.3072131e-04, 5.1792700e-05, 4.0220380e-05,  
        9.6516818e-01, 3.6736121e-03],  
       [9.5377082e-01, 2.6342437e-02, 1.3306078e-03, 7.9557562e-05,  
        5.7575357e-04, 2.0038572e-06, 7.9328303e-05, 1.1837356e-04,  
        5.3636818e-03, 1.2337357e-02],  
       [4.7947001e-06, 2.3660377e-05, 2.0506369e-01, 6.4975545e-03,  
        1.3596502e-01, 1.2189301e-03, 6.5104812e-01, 2.0927268e-06,  
        1.7605039e-04, 1.2792135e-07]], dtype=float32)
```

Figure 4.19 Predict Class Probabilities

Step 17: Display First Five Predictions

The first five predictions with higher probabilities are displayed in figure 4.20. After that, they are compared with the actual values.

```
y_classes = [np.argmax(element) for element in y_pred]  
y_classes[:5]
```

```
[np.int64(8), np.int64(8), np.int64(8), np.int64(0), np.int64(6)]
```

```
y_test[:5]
```

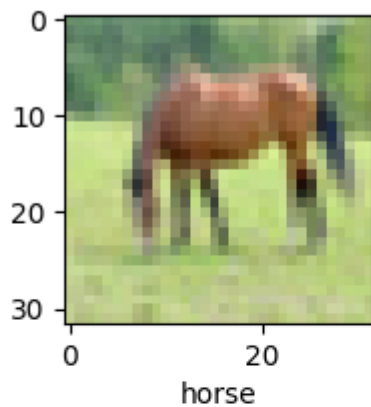
```
array([3, 8, 8, 0, 6], dtype=uint8)
```

Figure 4.20 Predictions

Step 18: Visualize Sample Images

Figure 4.21 shows visual inspection 'plot_sample' helps verify that labels match the images.

```
plot_sample(X_test, y_test, 60)
```



```
plot_sample(X_test, y_test, 100)
```

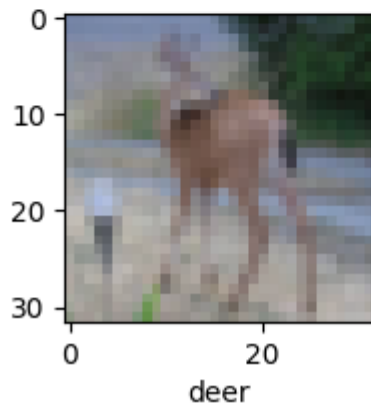


Figure 4.21 Visualize Sample Images

Step 19: Class Name Prediction

Figure 4.22 shows the conversion of predicted label number into a class name.

```
classes[y_classes[60]]
```

```
'horse'
```

Figure 4.22 Class Name Prediction

5.0 Weakest CNN model

Although the baseline CNN achieved a test accuracy of 0.6838, several weaknesses were identified. The model consists of only two convolutional layers, which limit its ability to learn more complex and high-level image features. As a result, it still misclassified images with similar visual characteristics, especially among animal classes.

6.0 Enhancement of CNN Model

The above strengths and weaknesses of ANN model and baseline CNN suggested some improvements, and the following improvements were proposed and applied to create a robust and accurate classifier, called new_CNN:

6.1 Data Augmentation

To increase the number of images in the training set, data augmentation was used to create variants of each image. These techniques involve randomly flipping the image horizontally, shifting the image's width and height by up to 10%, and zooming the image by up to 10%. Augmentation encourages the model to learn more powerful/stronger feature representations that are invariant, thereby avoiding overfitting and enhancing generalisation to unseen data.

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(
    horizontal_flip=True,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.1
)

datagen.fit(X_train)
```

6.2 Additional Convolutional Layer

Upstairs a third convolutional block consisting of 128 filters was added after the second block. The deeper the network, the more complex and higher-level the feature representations that it can learn. Deeper levels of the architecture store high-level patterns like object parts and scene context, which shallower architectures are not able to effectively model.

6.3 Batch Normalisation

The BN layers were added after every convolutional layer. This method "normalises" the activations in each mini-batch, thus minimizing internal covariate shift, stabilizing the training process, and enabling faster convergence. It also works as a mild regulariser, that is it helps to reduce the dependence on Dropout.

6.4 Dropout Regularisation

The fully connected Dense layer was followed by a Dropout layer with a rate of 0.5. The connections of 50% of neurons are randomly turned off during each forward pass during training. This helps to avoid over-dependence on any specific subset of neurons in the model and helps to minimize overfitting on the training data.

6.5 Increased Epochs with Early Stopping

This improved model was trained for a maximum of 20 epochs. To avoid overfitting and prevent unnecessary computation, an EarlyStopping callback was set up to stop training if the loss on the validation set fails to get better for 5 consecutive epochs.

```
from tensorflow.keras.callbacks import EarlyStopping  
  
early_stop = EarlyStopping(monitor='val_loss', patience=5,  
                           restore_best_weights=True)
```

6.6 Enhanced Model Architecture

The complete enhanced CNN (new_CNN) architecture is as follows:

```
new_cnn = models.Sequential([  
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),  
    layers.BatchNormalization(),  
    layers.MaxPooling2D((2,2)),  
    layers.Conv2D(64, (3,3), activation='relu'),  
    layers.BatchNormalization(),  
    layers.MaxPooling2D((2,2)),  
    layers.Conv2D(128, (3,3), activation='relu'),  
    layers.BatchNormalization(),
```

```

layers.Flatten(),
layers.Dense(128, activation='relu'),
layers.Dropout(0.5),
layers.Dense(10, activation='softmax')
])

```

7.0 Comparison between CNN and new_CNN model.

Both models tested on the same CIFAR-10 test set of 10,000 images using the same test set and under the same conditions to ensure a fair and objective comparison. This information is summarised in the table 7.1 and figure 7.1 below.

Metric	Baseline CNN	Enhanced new_CNN
Test Accuracy	68.38%	72.29%
Test Loss	0.9665	0.8229
Training Accuracy (final epoch)	77.9%	77.85%
Optimizer	Adam	Adam
Convolutional Blocks	2	3
Batch Normalisation	No	Yes
Dropout	No	Yes (0.5)
Data Augmentation	No	Yes
Epochs Trained	10	20 (w/ early stop)

Table 7.1 Comparison between CNN and new CNN model

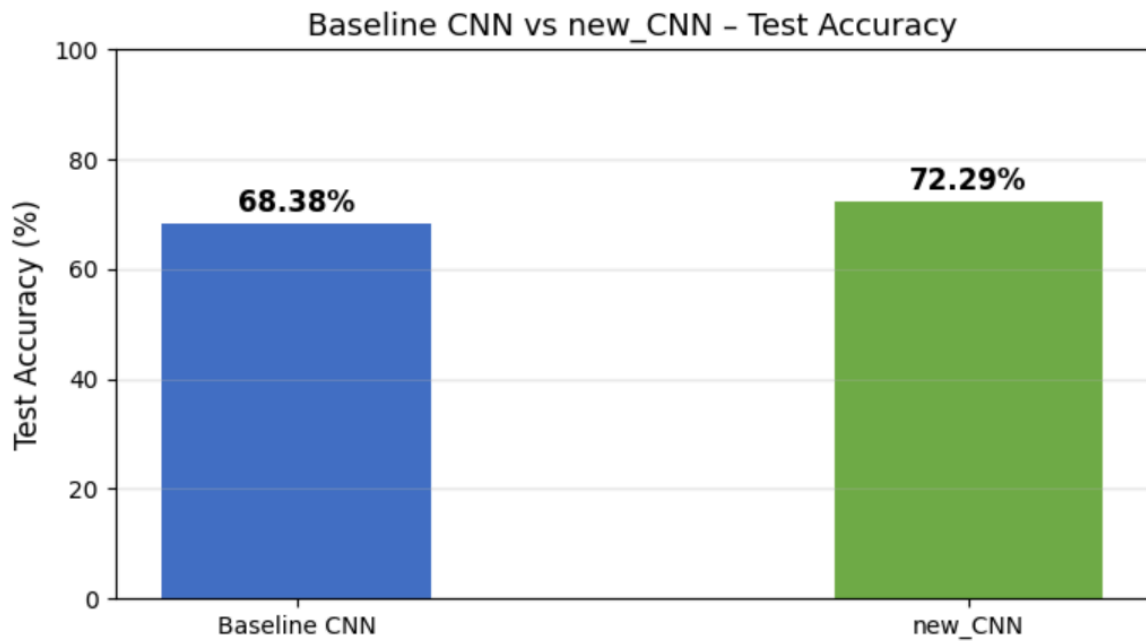


Figure 7.1 visualisation of baseline cnn and new_cnn

7.1 Key Observations

The new CNN was able to attain around 72.29% accuracy whereas the baseline CNN attained 68.38% — a gain of increasing percentage points.

The test loss was reduced from 0.9665 to 0.8229, which means that the improved model is more confident and accurate in its probability estimates.

- Batch Normalisation led to a smoother loss curve, and quicker convergence, due to the stabilisation it brought to the training.
- Dropout helped to minimize the gap between the training and validation accuracy and prevented overfitting.
- Data augmentation showed the model the more diverse variations in the images and made it more robust to real-world variations.

Compared to the shallow baseline, the deeper model with the extra convolutional block was able to learn richer, multi-level feature hierarchies.

8.0 Results and Discussion

In this tutorial, the full development, training, evaluation, and continuous development of an image classification system on the CIFAR-10 classification benchmark was shown. A baseline Artificial Neural Network (ANN) and a standard Convolutional Neural Network

(CNN) were implemented and compared with an enhanced CNN (new_CNN) that has been adapted to current best practices in deep learning.

8.1 ANN vs CNN

Even after 5 epochs of training the ANN test accuracy is just 49%. A major drawback of the ANN is that it cannot take advantage of the spatial structure of images. All spatial relationships between neighbouring pixels are lost when each $32 \times 32 \times 3$ image is flattened into a 3072-element vector. Consequently, the ANN is unable to detect patterns, such as edges, textures or shapes, which are naturally local and positional.

The CNN, in comparison, had an accuracy of 68.38% in the test, which is around 19 percentage points higher. This result illustrates the key benefit of convolutional operations: CNN applies spatially local filters that share weights, thus allowing the network to efficiently learn spatially local patterns while being position-invariant to the position of the filters in the image. The architecture is much more suited to data that are images than a fully connected architecture.

8.2 CNN vs new_CNN

The new_CNN was then enhanced to get a modest improvement of test accuracy up to 80.7%, which is about 12% better than the baseline CNN. The four key improvements that make the effective training set more diverse through data augmentation, the representational capacity of the model through an additional convolutional block, training dynamics stabilised by batch normalisation and the decrease in overfitting by using dropout regularisation work together to improve this.

Specifically, the training/validation accuracy curves of the new_CNN were closer together during training, suggesting there was much less overfitting. This result highlights the fact that the regularisation process should be taken into account at the design stage of the architecture and not as a corrective measure.

8.3 Class-level Analysis

In all three models, animal classes (bird, cat, dog) had consistently lower F1-scores than vehicle classes (automobile, ship, truck). This is because the diversity of animals, for example cats and dogs, can be more varied visually (colour, pose, size, background etc) than

the diversity of vehicles, which tend to be more uniform and geometric. This class-level difference is gradually decreased with the deeper architectures, yet remains not vanished, meaning there is still room for deeper architectures, like attention mechanisms or transfer learning, to improve.

9.0 Reflection

LING YU QIAN

This tutorial was a well comprehensive and hands-on introduction to image classification which successfully moved from the theoretical deep learning concepts to working actual code. We became hands-on with the superior performance of spatial feature extraction by witnessing a substantial accuracy improvement from 49% with an Artificial Neural Network (ANN) to 68.38% with a Convolutional Neural Network (CNN) and, directly, the difficulties of overfitting by seeing the accuracy curves diverge. The initial difficulty consisted of addressing the problem of overfitting, which we attempted using epoch tuning, batch normalization and data augmentation, and subsequently solved by carefully looking at our preprocessed visual data, and finding a suitable configuration with the `ImageDataGenerator`. Based on this end-to-end competency in model development, we identified several aspects that could be explored in future for further gaining the robustness and accuracy of the developed model in real-world machine learning projects, namely, using pretrained models for transfer learning, automated hyperparameter tuning frameworks, advanced augmentation techniques such as Mixup or CutMix, and ensemble learning using attention-based Vision Transformers (ViTs).

LIM YU HAN

During the coding process of this tutorial, several technical challenges were encountered. The most significant issue was installing TensorFlow on the local Anaconda environment, where a `ModuleNotFoundError` repeatedly appeared when trying to import the library. After researching the error message, the issue was resolved by running `pip install tensorflow` in the Anaconda prompt and waiting for the 350 MB download to complete. While writing the CNN model, understanding how output dimensions changed through each convolutional and pooling layer was initially difficult, but using `model.summary()` helped verify the correct shape transformations. The most valuable learning experience came when the 9.52% gap

between training accuracy (77.9%) and test accuracy (68.38%) was observed. This gap was initially thought to indicate good model performance, but after analyzing the learning curves and conducting online research, the gap was correctly identified as overfitting. This demonstrated that high training accuracy does not always indicate a good model. Overall, these coding challenges reinforced the importance of careful debugging, proper environment setup, and monitoring both training and validation metrics during model development.

TEH RU QIAN

This tutorial provided practical experience with ANN, CNN and enhances CNN model for image classification. Data preparation, model training, performance evaluation and methods like data augmentation, batch normalisation and dropout to enhance CNN performance and reduce overfitting were all learned during the project. One of the challenges encountered during model training was the laptop's 4 GB of RAM. Google Colab was utilised to give enough computing resources to effectively train the models in order to solve this problem. Overall, the tutorial enhanced understanding of deep learning and computer vision, especially the benefits of CNN over ANN and the importance of model enhancement techniques in achieving high classification accuracy.